

Simultaneous multi-threading (hyperthreading)

For what follows, I referred primarily to J. Stokes article, “Introduction to Multithreading, Superthreading and Hyperthreading”, available at:

arstechnica.com/paedia/h/hyperthreading/hyperthreading-1.html

To understand hyperthreading (which Intel is incorporating into its next generation of processors), we have to recall two key concepts from earlier in the course: multitasking and superscalar execution.

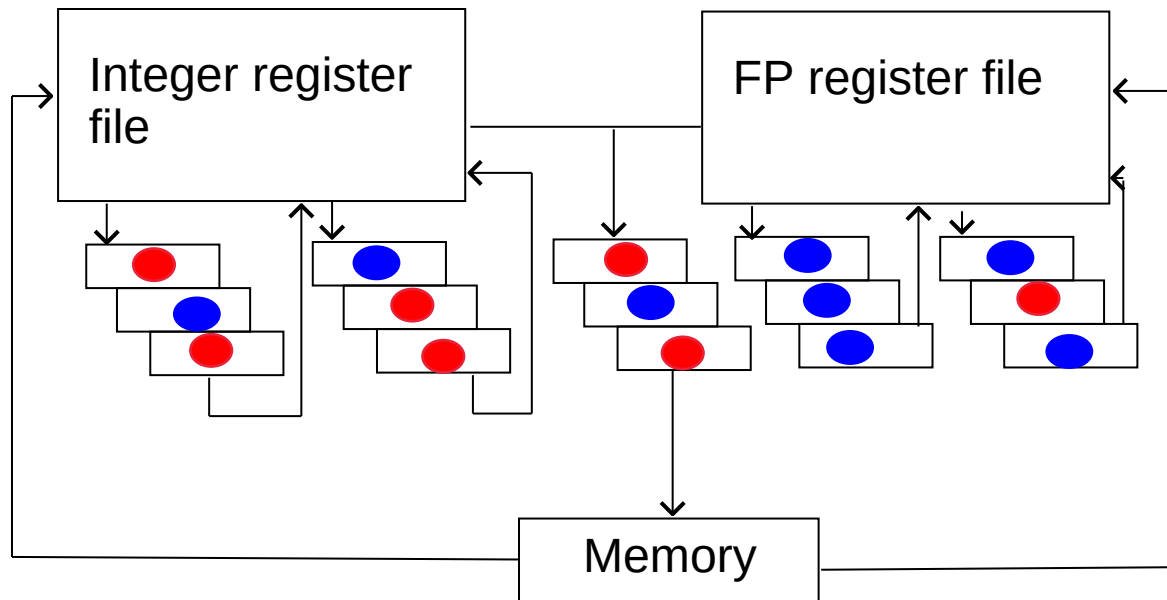
Multiple process execution

In our discussion of operating systems, we said that in “multi-tasking” operating systems only one process was ever **actually executing** on the CPU at any given time. The users (and the processes themselves) are **tricked** into thinking that more than one process is executing by judicious **task-switching** and **time-slicing**. The fundamental goal of multi-tasking systems was to **maximize CPU utilization** to make sure that if a process had to stall waiting for some resource, some other process would be allowed to run.

We also looked (briefly) at multi-processor systems, in which a computer contains more than one CPU. In such systems, each CPU can be executing a different program, so multiple processes can actually execute.

Superscalar architecture

The core of superscalar architecture was the use of **multiple, pipelined functional units**. More than one instruction was allowed to be issued at a time, provided instructions could execute on different functional units inside the CPU.



A problem observed in benchmark studies is that, on average, even the smartest compilers can not organize code to fully utilize all of the resources in the CPU. Studies seem to show, for example, that on average, not more than 2.5 instructions from a program can be issued per clock cycle. This is why the Pentium family limits instruction issue to 3 instructions per cycle.

Unfortunately, this causes “bubbles” in the pipelines—cycles during which the various functional units of the CPU are doing nothing.

Solution: hyperthreading

The solution employed by the newest Pentium class (Xeon) processor is called **simultaneous multi-threading** or **hyperthreading** (for our purposes, a “thread” and a “process” can be considered the same thing). The goal in hyperthreading is to allow the CPU to issue instructions *from more than one process simultaneously*.

If, for example, it is only possible to issue 2.5 instructions/cycle from a given program, then if we let the CPU issue instructions from *two* programs, we would get on average 5 instructions/cycle issued and reduce the number of pipeline bubbles.

Hyperthreading in the Xeon

The Pentium 4 Xeon allows two threads to execute simultaneously on the CPU. In order to accomplish this (very impressive) feat, the Xeon breaks CPU resources into three classes: replicated, partitioned and shared.

Replicated resources are those that must exist twice. Every instruction needs its own program counter, register rename logic and translation look-aside buffer (for paging), for example.

Partitioned resources are basically scheduling queues (what we called the “instruction window”). They are partitioned to guarantee that each executing process has access to an equal share of the queue, however the actual elements in the queue are dynamically allocated. The goal is maximum flexibility without letting either process “hog” the queue.

Shared resources are everything else. The most important shared resource are the register file and functional units. (Most important since it was the optimization of these resources that motivated hyperthreading in the first place). There is the potential for a bottleneck here. If, for example, two floating-point intensive processes are trying to execute extensively, then they will block each other (losing our parallelism) and let the integer functional units go to waste.

The cache is also treated as a shared resource. Note that hyperthreading *eliminates the cache coherence problem* from SMP systems, since our two processes can communicate through the same local cache. Ideally, the cache should dynamically balance to allow each process to have its fair share. In the worst case, we may get one process hogging the cache, or some thrashing as cache lines are repeatedly swapped in by each process.